Comprehensive Fuzzy Transformer Study

A controlled comparison of membership curves, Mamdani rule systems, fuzzy placement, attention mechanisms, encoder depth, input windows, and time-series Transformer backbones

Study scale	Confirmed winner	Held-out RMSE	Main caution
72 declared cases 64 unique models	W10, 1 layer, 7 rules parabolic -> triangular	**1.4436 +/- 0.0450**	Persistence is better (1.3727)

Executive verdict

The broad experiment selected a **standard custom Transformer with one encoder layer, a 10-observation input window, and a seven-rule Mamdani layer**. The Mamdani antecedents are parabolic, the consequents are triangular, and fuzzy inference is applied after the feed-forward sublayer. Its confirmed validation RMSE is **1.6512 +/- 0.0775**. It narrowly leads the nine confirmed candidates, but its uncertainty overlaps with them.

On the 374-example held-out test, the three-seed mean RMSE is **1.4436**. This is 5.17% worse than persistence, 1.21% worse than the archived API Transformer run, and 2.65% worse than the archived single-seed FANTF run. It is 32.60% better than the reproduced BCM artifact. Historical single-seed results are context, not a statistical comparison with the three-seed selected model.

Supported conclusion: the comprehensive search identifies a sensible fuzzy Transformer candidate and shows which design choices generalize across validation folds. It does not show that extra fuzzy machinery automatically beats simple forecasting baselines.

1. Research question and evaluation target

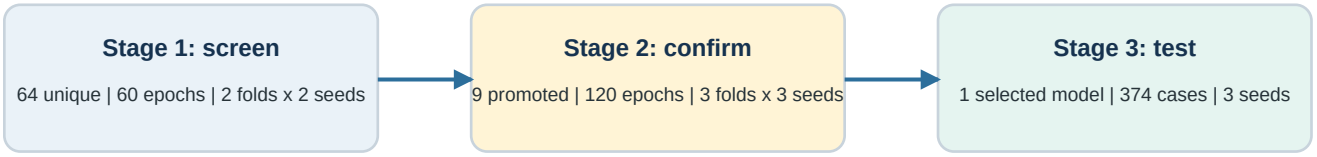
The project replaces the BCM association-learning path in the supplied ieRSPOP workflow with an explicit Transformer forecast model. The target is next-step volatility: the model receives a chronological window of already observed volatility values and predicts the next value.

Questions answered

#	Research question
1	Which antecedent and consequent membership-curve pair works best?
2	How many Mamdani rules, encoder layers, and past observations are useful?
3	Should Mamdani inference follow attention, the feed-forward block, or both?
4	Does FANTF noise or deterministic Gaussian-distance attention improve standard attention?
5	How do lightweight PatchTST-style and iTransformer-style backbones behave on this small univariate series?

Dataset and scored protocol

Item	Setting
Available windows	860 next-step examples
Held-out test	374 examples; first recorded MATLAB window 487
Prediction order	Predict first, reveal target, then update from replay window
Normalization	Fit using observations available before the scored target
Primary metric	RMSE; MAE, Pearson r, and predictive R-squared also reported
Simple baseline	Persistence: predict the most recent observed value

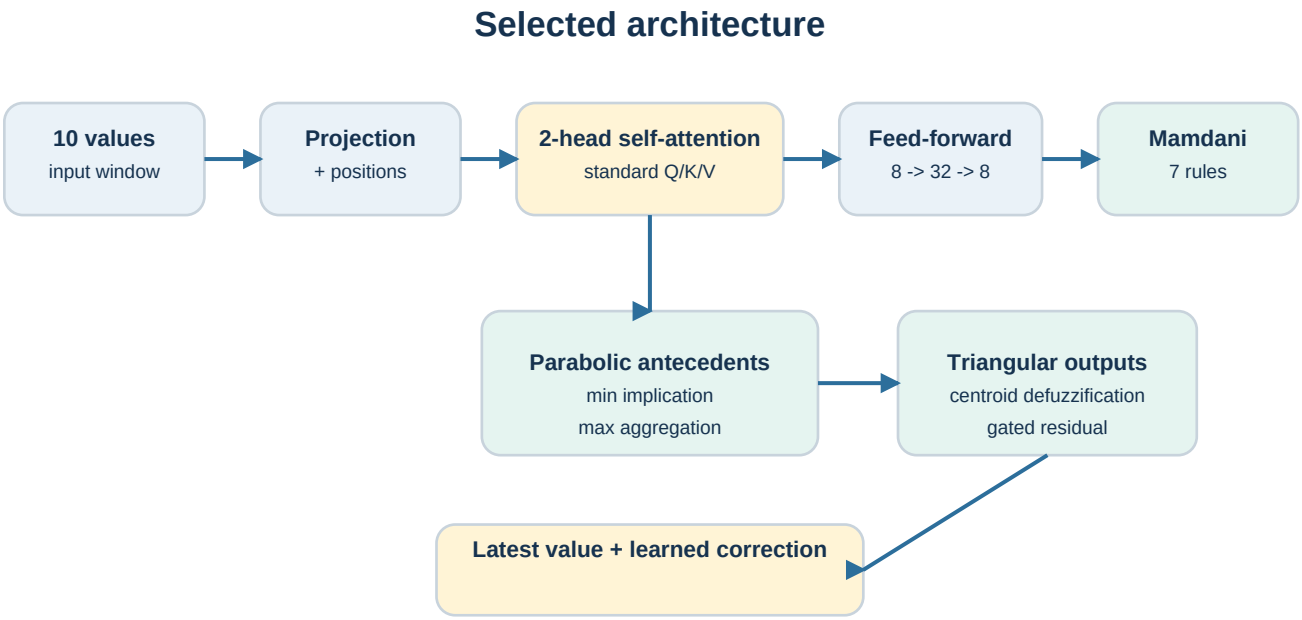


The held-out test is consulted only after validation-based selection.

This staged design limits test-set leakage. Screening is intentionally broad and cheaper; confirmation is stronger; only the selected confirmation winner reaches the held-out test.

2. Implemented architectures

Selected model



Every Transformer component remains explicit: Q/K/V projections, head splitting, scaled dot-product attention, softmax, residual paths, layer normalization, and the position-wise feed-forward network. No prebuilt PyTorch Transformer encoder or multi-head attention module is used.

Variant	Tokenization / attention	Purpose in this study
Custom Transformer	One time point per token; standard dot-product attention	Capacity-controlled base encoder
FANTF-style	Learned Gaussian perturbation added to attention scores	Test stochastic fuzzy attention and delta
Deterministic fuzzy	Gaussian membership from learned Q/K distance widths	Test fuzziness without random score noise
PatchTST-style	Overlapping length-4 patches, stride 2, channel independent	Test patch tokens at a 20-step window
iTransformer-style	Variables become tokens; temporal history is projected	Test inverted tokenization; one token in univariate data

The PatchTST-style and iTransformer-style models are lightweight project adaptations built from the same custom encoder, not reproductions of the authors' full official training recipes. The iTransformer-style comparison is structurally disadvantaged here because a univariate series creates only one variable token, leaving no cross-variate attention to learn.

3. Mamdani layer and membership functions

Each hidden feature has an ordered low-to-high fuzzy rule base. Hidden values are bounded with tanh, fuzzified against learnable centers and widths, and passed through Mamdani minimum implication and maximum aggregation. Centroid defuzzification creates a crisp update that enters through a learned residual gate.

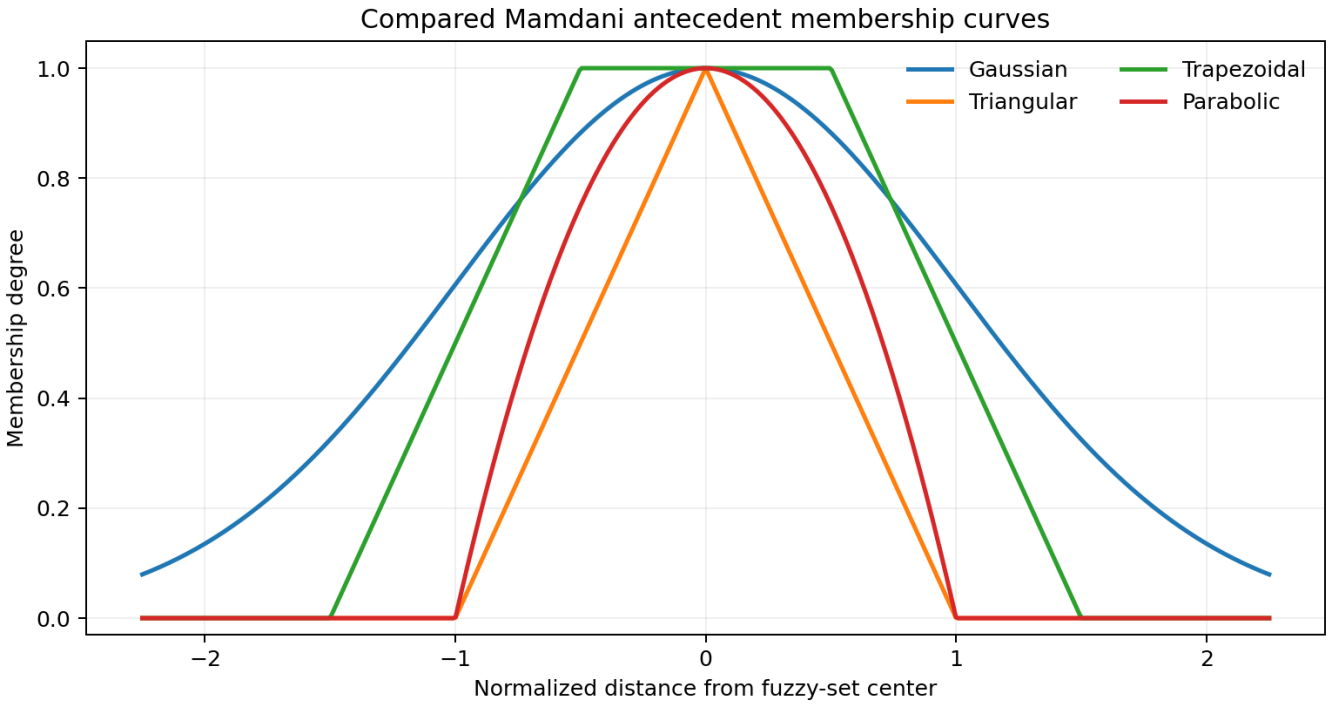


Figure 1. Four differentiable membership families used for antecedents and consequents.

Curve	Membership degree for normalized distance d	Property
Gaussian	exp(-0.5 d^2)	Smooth, nonzero tails
Triangular	max(1 - |d|, 0)	Compact and linear
Trapezoidal	clip(1.5 - |d|, 0, 1)	Flat core with linear shoulders
Parabolic	max(1 - d^2, 0)	Smooth compact support

The comprehensive study tests all 16 antecedent/consequent pairings. Centers and antecedent widths are learnable. Consequent shapes are fixed on a discretized output universe, keeping the Mamdani rules interpretable.

4. Experiment matrix

Family	Settings	Declared cases
Core factorial	Mamdani vs FANTF+Mamdani x rules 3/5/7 x layers 1/2 x windows 5/10/20	36
Curve pairs	4 antecedent curves x 4 consequent curves	16
Fuzzy placement	After attention, after feed-forward, both	3
Encoder depth	1, 2, 3 layers	3
Sequence length	5, 10, 20, 30 observations	4
FANTF delta	0.1, 0.25, 0.5, 1.0	4
Attention type	Standard, FANTF, deterministic Gaussian fuzzy	3
Backbone	Transformer, PatchTST-style, iTransformer-style	3
Total	72 declared; duplicate base settings cached	64 unique

Training budgets

Phase	Epochs	Folds	Seeds	Observation count per configuration
Screening	60	2	42, 52	4 fold-seed results
Confirmation	120	3	42, 52, 62	9 fold-seed results
Held-out	120	374 test cases	42, 52, 62	3 complete prequential paths

Promotion rule: confirm the winner from each experiment family plus the overall top five unique screening configurations. The primary ranking statistic is mean RMSE.

5. Screening results

Experiment family	Screening winner	RMSE	SD
Core Factorial	Mamdani, 3 rules, 1 layer(s), window 10	1.5904	0.0476
Sequence Length	window 10	1.5904	0.0476
Curve Pairs	parabolic -> gaussian	1.6099	0.0470
Attention Type	FANTF stochastic	1.6126	0.0539
Fantf Delta	delta 0.1	1.6141	0.0530
Encoder Depth	1 encoder layer(s)	1.6225	0.0542
Fuzzy Placement	after_feedforward	1.6225	0.0542
Backbone	itransformer	1.6507	0.0362

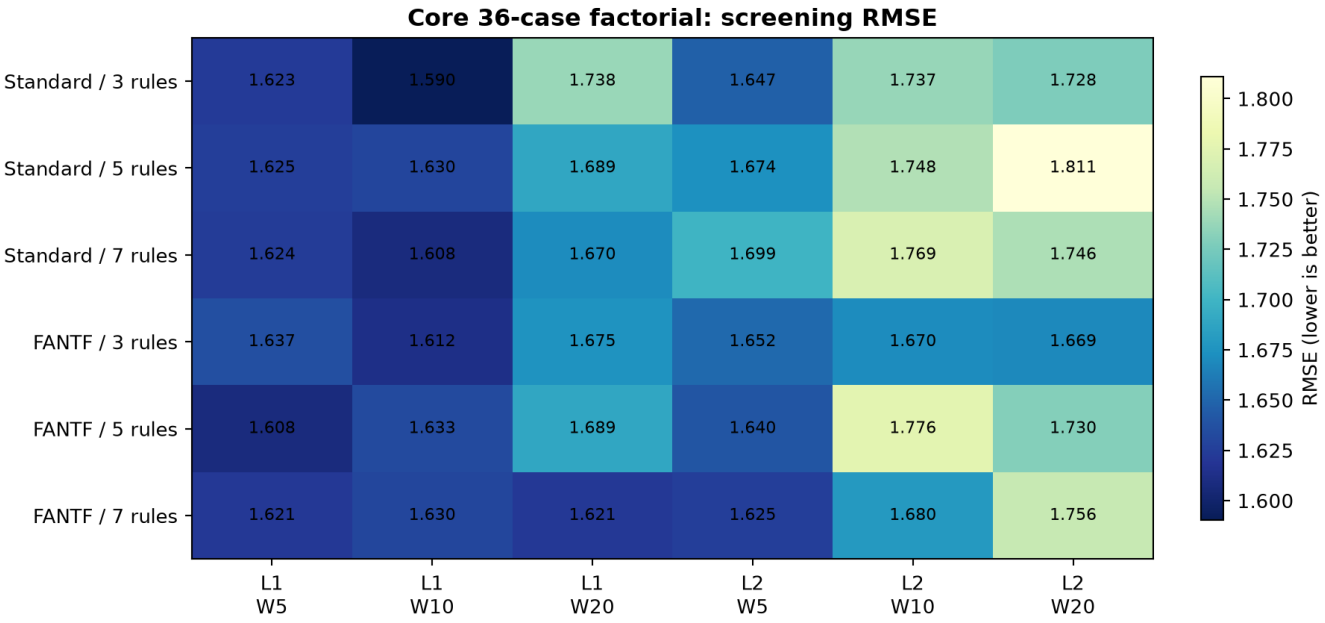


Figure 2. The core screen favors one layer and a 10-observation window. More rules or a second layer do not produce a uniform improvement. Screening values are selection evidence, not final test claims.

6. Curve-pair comparison

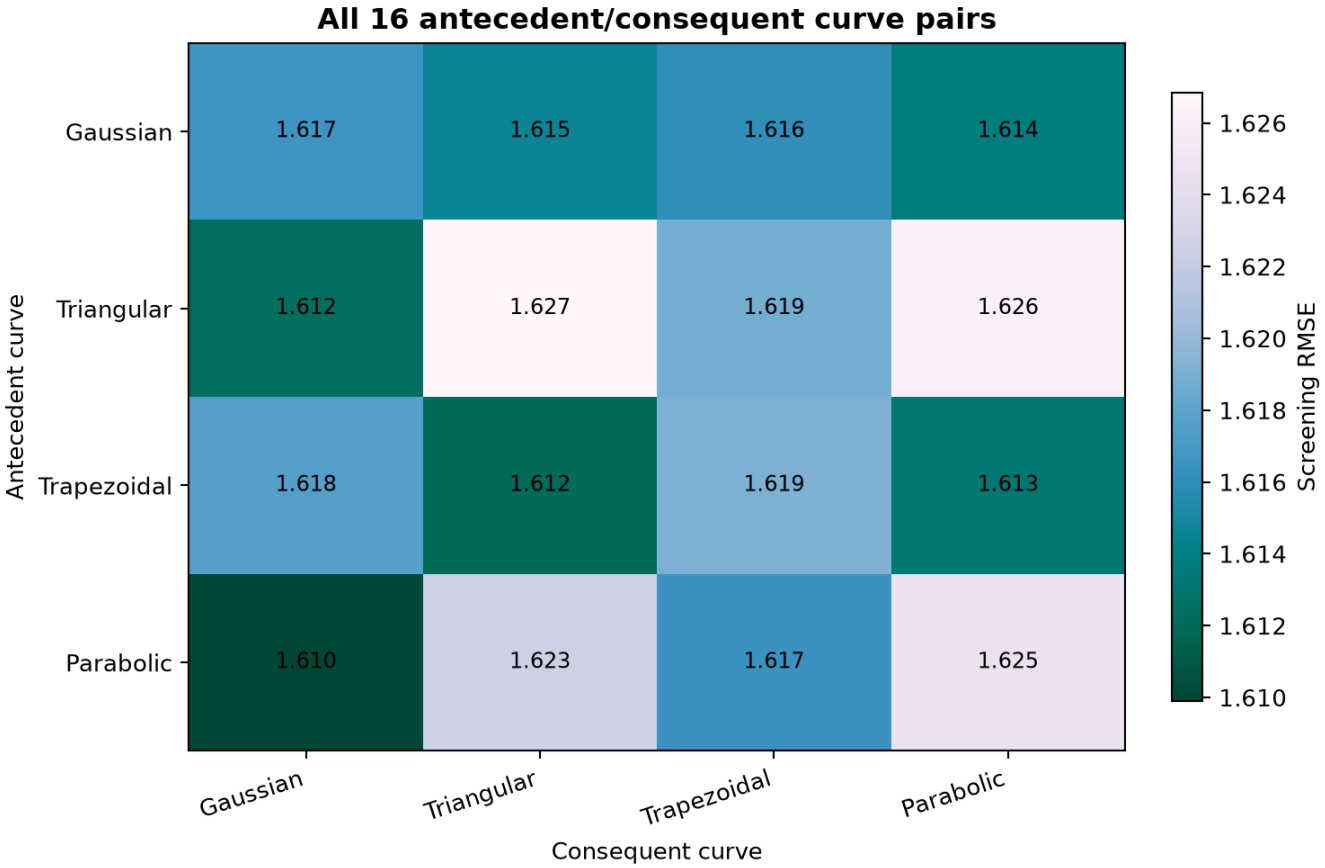


Figure 3. Screening RMSE for every antecedent/consequent curve pair. Parabolic antecedents with Gaussian consequents lead this short screen, while the earlier parabolic/triangular design remains competitive.

Rank	Antecedent	Consequent	RMSE	SD
1	parabolic	gaussian	1.6099	0.0470
2	trapezoidal	triangular	1.6120	0.0427
3	triangular	gaussian	1.6123	0.0456
4	trapezoidal	parabolic	1.6131	0.0449
5	gaussian	parabolic	1.6137	0.0451
6	gaussian	triangular	1.6146	0.0460

Important reversal: parabolic -> Gaussian wins the cheaper curve screen but ranks sixth during full confirmation. The final winner returns to parabolic -> triangular with seven rules and a longer window. This is exactly why a second confirmation stage is necessary.

7. Placement, attention, depth, window, and backbone sweeps

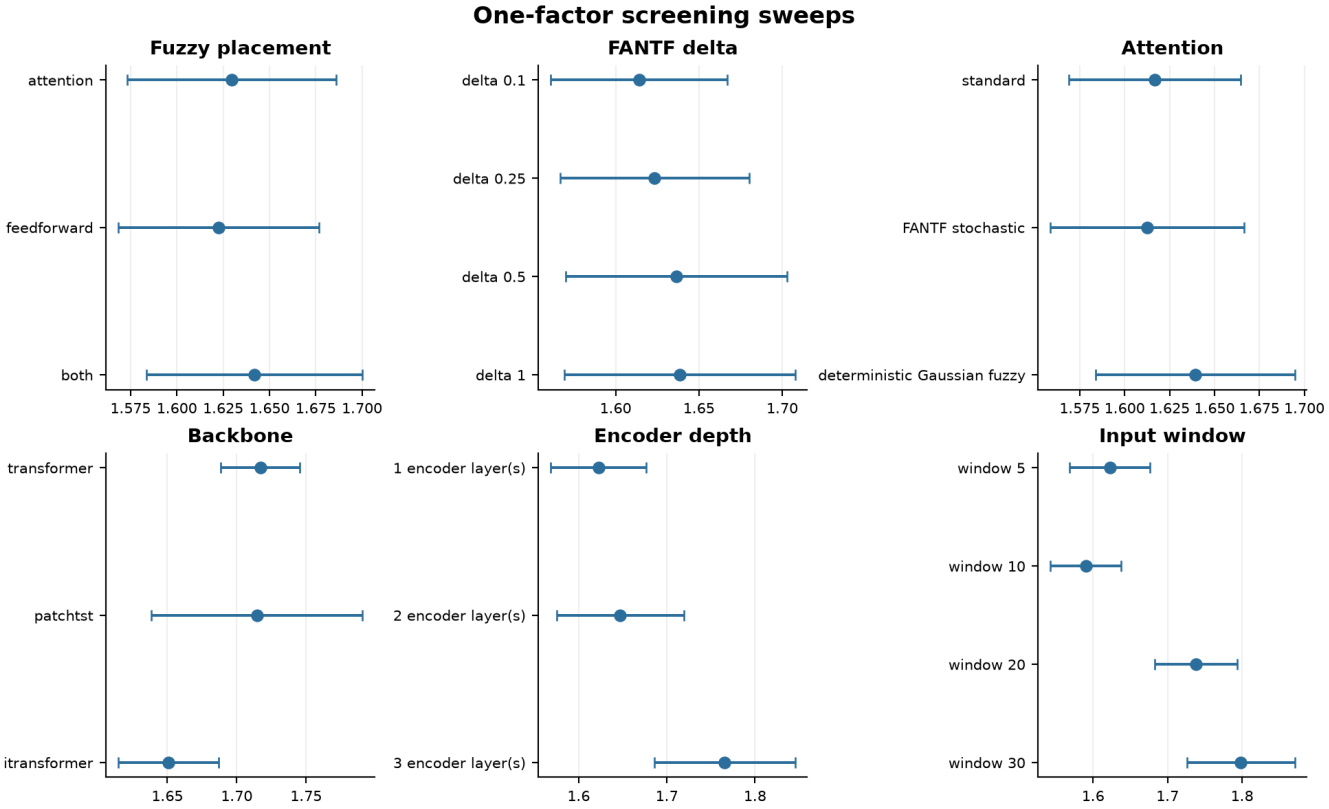


Figure 4. One-factor screening sweeps. Error bars show variation across two folds and two seeds.

Factor	Observed screening pattern
Fuzzy placement	After feed-forward leads; applying Mamdani twice adds parameters without a reliable gain.
FANTF delta	0.1 is the strongest tested noise scale; larger perturbations do not help consistently.
Attention	FANTF leads the short attention-only screen, but FANTF-only ranks eighth after confirmation.
Depth	One encoder layer is preferable for this 860-example series.
Window	Ten observations lead both screening and final selection.
Backbone	iTransformer-style wins the short backbone screen but finishes last in full confirmation.

The iTransformer-style result should not be generalized to multivariate forecasting. With one input variable it has one variable token, so the cross-variate attention mechanism that motivates iTransformer cannot operate as intended.

8. Full confirmation

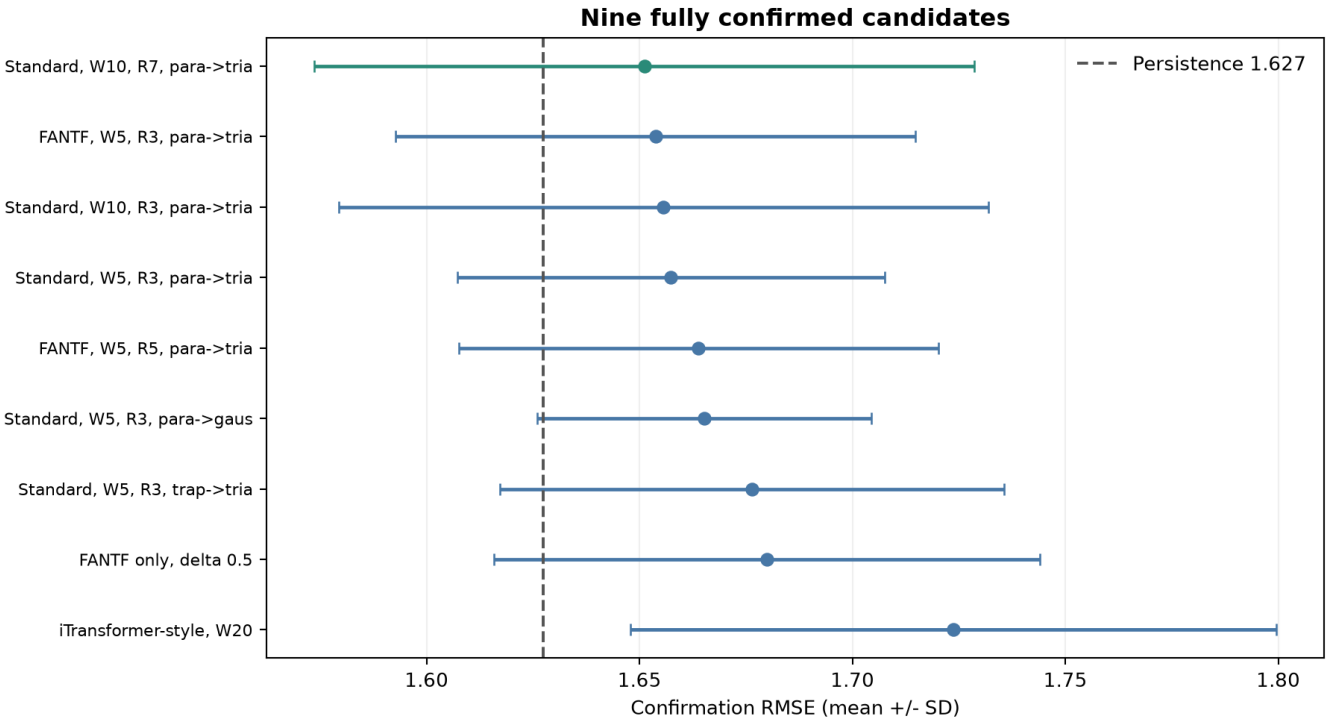


Figure 5. Nine promoted configurations rerun with 120 epochs, three chronological folds, and three seeds. The dashed line is persistence on the same validation folds.

Rank	Configuration	Params	RMSE	MAE	r
1	standard W10 R7 para->tria	1113	1.6512	1.2896	0.7937
2	fantf W5 R3 para->tria	1010	1.6538	1.2647	0.7888
3	standard W10 R3 para->tria	1049	1.6556	1.2796	0.7920
4	standard W5 R3 para->tria	1009	1.6574	1.2656	0.7893
5	fantf W5 R5 para->tria	1042	1.6638	1.2664	0.7878
6	standard W5 R3 para->gaus	1009	1.6652	1.2767	0.7880
7	standard W5 R3 trap->tria	1009	1.6764	1.2766	0.7852
8	FANTF-only W5	954	1.6799	1.2890	0.7808
9	iTransformer-style W20	1073	1.7237	1.3359	0.7758

The winner's RMSE advantage over second place is only 0.16%. Standard-deviation intervals overlap strongly, so the correct wording is 'selected by mean RMSE,' not 'proven superior.' The winner is also 1.46% worse than persistence on confirmation folds.

9. Held-out test results

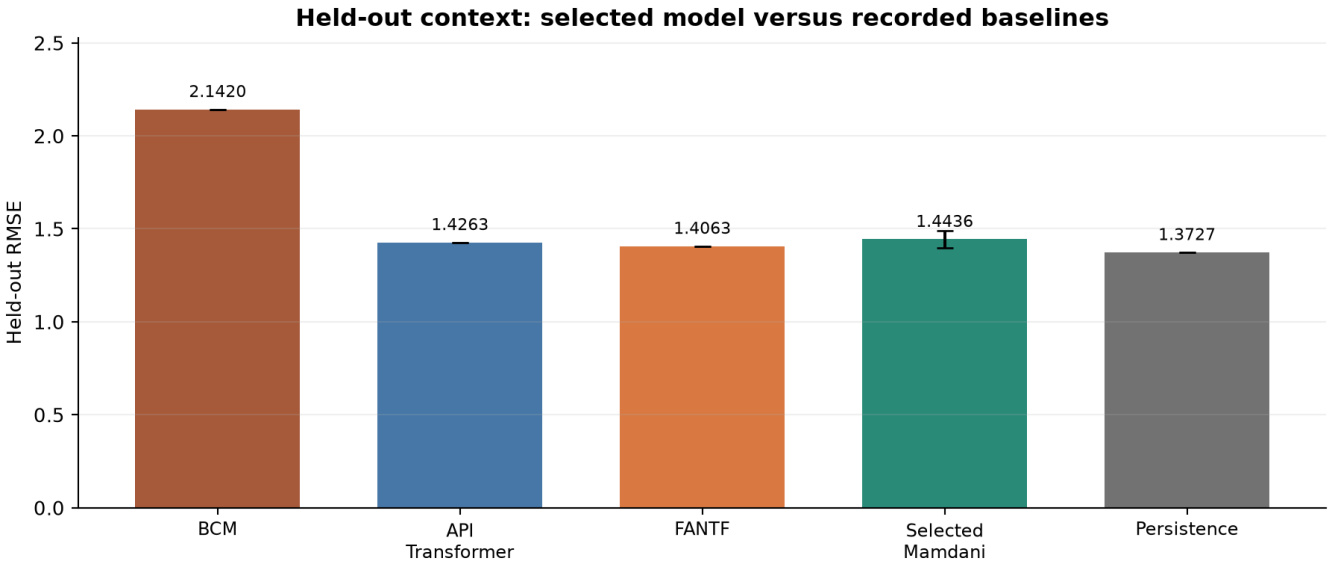


Figure 6. Historical BCM, API Transformer, and FANTF artifacts are single recorded runs. The selected Mamdani bar reports the mean and SD of three seeds; persistence is deterministic.

Metric	Selected mean	Seed SD	Persistence	Interpretation
RMSE	1.4436	0.0450	1.3727	Persistence lower
MAE	1.0811	0.0376	0.9158	Persistence lower
Pearson r	0.9431	0.0032	0.9476	Persistence higher
Predictive R2	0.8842	0.0073	0.8954	Persistence higher

Seed	RMSE	MAE	Pearson r	Predictive R2
42	1.5056	1.1214	0.9386	0.8742
52	1.3999	1.0310	0.9459	0.8912
62	1.4255	1.0909	0.9449	0.8872

The selected model uses 1113 trainable parameters. It beats the reproduced BCM artifact by 32.60% RMSE, but does not improve on persistence or the archived single-seed Transformer artifacts. The code also carries a separate published/reference ieRSPOP++ RMSE of 1.8045; that value is not substituted for the reproduced BCM artifact of 2.1420.

10. Forecast behavior

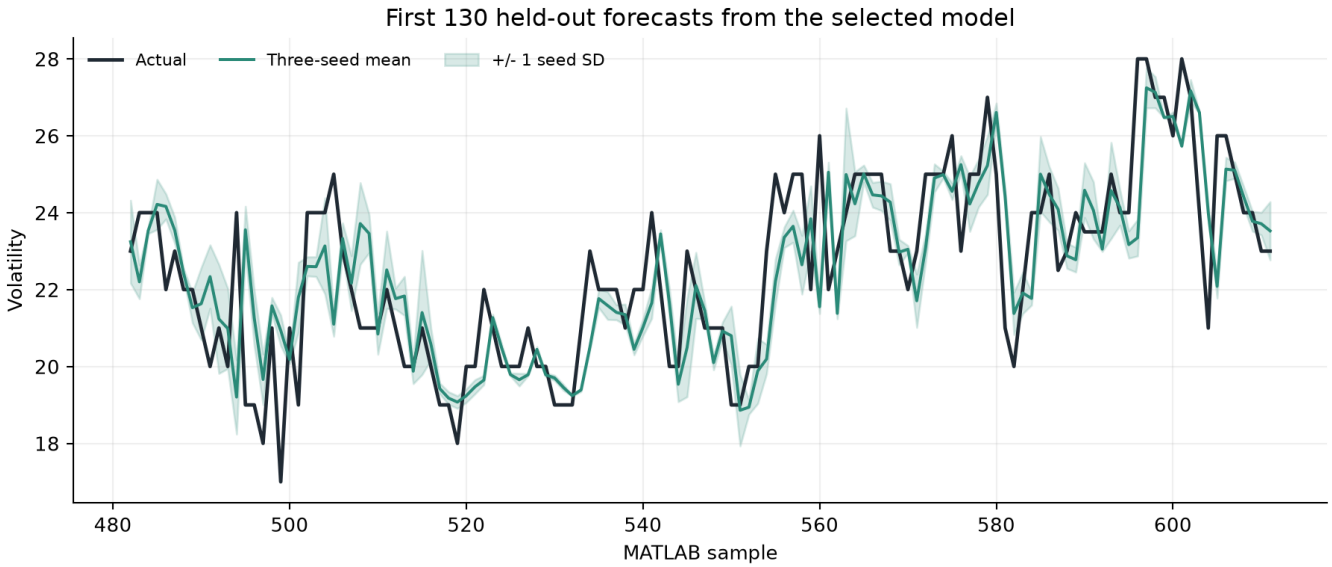


Figure 7. The selected model follows broad volatility movement, while seed-to-seed spread is usually small. Large local changes remain difficult, which helps explain why copying the latest value is strong.

Why persistence is hard to beat

The volatility series is locally smooth and the forecast horizon is one step. Persistence has no estimation variance and directly exploits that smoothness. The Transformer begins from persistence through a residual connection, but a learned correction can still add error when the available 860 examples do not support a stable nonlinear adjustment.

What the Transformer still contributes

The experiment demonstrates an interpretable, trainable replacement for the BCM association path; quantifies the value of fuzzy design choices; and provides a framework that can benefit from additional variables, longer histories, or more data. The negative result against persistence is useful evidence, not a failed experiment.

11. Limitations and threats to validity

Issue	Impact	Mitigation / next step
Small univariate dataset	Deep and multivariate backbones cannot show their intended strengths.	Add exogenous variables or evaluate on a multivariate benchmark.
Broad model search	Even validation-only screening can favor noise.	Use nested rolling validation or a second untouched dataset.
Two-seed screening	Early rankings are noisy.	The family winners and top cases were rerun with three seeds and three folds.
Historical baseline mismatch	API and FANTF context uses one recorded seed, while the selected model uses three.	Rerun all historical learned baselines with the same three seeds.
Style adaptations	PatchTST/iTransformer results are not official-paper replications.	Use official repositories and benchmark training recipes for publication claims.
iTransformer degeneracy	One variable means one attention token.	Retest with genuinely multivariate inputs.
No statistical test	Overlapping SDs prevent a strong superiority claim.	Use paired errors across seeds/folds and a suitable paired test.

Claims that are supported

#	Supported statement
1	All announced fuzzy/Transformer categories were implemented and evaluated under a staged protocol.
2	The 10-window, seven-rule parabolic/triangular Mamdani Transformer has the lowest confirmed mean RMSE among promoted candidates.
3	More depth, stronger FANTF noise, or additional fuzzy placements do not reliably improve this dataset.
4	Persistence remains the strongest held-out forecast in the current evidence.

12. Conclusion and professor-ready explanation

The comprehensive experiment did not simply tune one fuzzy curve. It separated the main design axes, screened 64 unique models, confirmed nine finalists, and evaluated one selected model on the held-out test. The winning architecture is compact: one standard-attention encoder layer, a 10-value history, seven parabolic-input/triangular-output Mamdani rules, and fuzzy inference after the feed-forward block.

Suggested 45-second explanation

We built the Transformer internally from basic PyTorch layers, then added a differentiable Mamdani system inside each encoder block. We tested 72 declared combinations covering rule counts, membership curves, fuzzy placement, attention type, model depth, history length, and PatchTST/iTransformer-style tokenizations. To avoid choosing from the test set, we screened on chronological validation folds, fully confirmed nine finalists, and tested only the winner. The selected model uses one layer, ten past values, and seven rules with parabolic antecedents and triangular consequents. It clearly improves over the reproduced BCM result, but persistence is still better, so our conclusion is that the fuzzy Transformer is a viable learned replacement rather than the best overall forecast on this small series.

Recommended next experiment

Freeze the selected architecture, rerun persistence, the API Transformer, FANTF-only, and the selected Mamdani model across at least five identical seeds, and compare paired errors on the same chronological folds. Then repeat on a genuinely multivariate volatility dataset where PatchTST and iTransformer can operate in their intended regime.

Appendix A. Complete screening summary

Family	Configuration	Params	RMSE	SD
attention type	FANTF stochastic	954	1.6126	0.0539
attention type	standard	953	1.6168	0.0477
attention type	deterministic Gaussian fuzzy	955	1.6393	0.0554
backbone	itransformer	1073	1.6507	0.0362
backbone	patchtst	1009	1.7145	0.0762
backbone	transformer	1073	1.7170	0.0284
core factorial	Mamdani, 3 rules, 1 layer(s), window 10	1049	1.5904	0.0476
core factorial	FANTF+Mamdani, 5 rules, 1 layer(s), window 5	1042	1.6082	0.0497
core factorial	Mamdani, 7 rules, 1 layer(s), window 10	1113	1.6084	0.0403
core factorial	FANTF+Mamdani, 3 rules, 1 layer(s), window 10	1050	1.6120	0.0554
core factorial	FANTF+Mamdani, 7 rules, 1 layer(s), window 20	1194	1.6211	0.0197
core factorial	FANTF+Mamdani, 7 rules, 1 layer(s), window 5	1074	1.6212	0.0468
core factorial	Mamdani, 3 rules, 1 layer(s), window 5	1009	1.6225	0.0542
core factorial	Mamdani, 7 rules, 1 layer(s), window 5	1073	1.6238	0.0525
core factorial	Mamdani, 5 rules, 1 layer(s), window 5	1041	1.6249	0.0503
core factorial	FANTF+Mamdani, 7 rules, 2 layer(s), window 5	2067	1.6254	0.1073
core factorial	Mamdani, 5 rules, 1 layer(s), window 10	1081	1.6299	0.0442
core factorial	FANTF+Mamdani, 7 rules, 1 layer(s), window 10	1114	1.6303	0.0502
core factorial	FANTF+Mamdani, 5 rules, 1 layer(s), window 10	1082	1.6327	0.0404
core factorial	FANTF+Mamdani, 3 rules, 1 layer(s), window 5	1010	1.6365	0.0664
core factorial	FANTF+Mamdani, 5 rules, 2 layer(s), window 5	2003	1.6401	0.1037
core factorial	Mamdani, 3 rules, 2 layer(s), window 5	1937	1.6471	0.0722
core factorial	FANTF+Mamdani, 3 rules, 2 layer(s), window 5	1939	1.6521	0.0794
core factorial	FANTF+Mamdani, 3 rules, 2 layer(s), window 20	2059	1.6690	0.0380
core factorial	FANTF+Mamdani, 3 rules, 2 layer(s), window 10	1979	1.6705	0.0879
core factorial	Mamdani, 7 rules, 1 layer(s), window 20	1193	1.6705	0.0721
core factorial	Mamdani, 5 rules, 2 layer(s), window 5	2001	1.6736	0.0942
core factorial	FANTF+Mamdani, 3 rules, 1 layer(s), window 20	1130	1.6754	0.0594
core factorial	FANTF+Mamdani, 7 rules, 2 layer(s), window 10	2107	1.6796	0.0606
core factorial	FANTF+Mamdani, 5 rules, 1 layer(s), window 20	1162	1.6893	0.0510
core factorial	Mamdani, 5 rules, 1 layer(s), window 20	1161	1.6893	0.0696
core factorial	Mamdani, 7 rules, 2 layer(s), window 5	2065	1.6991	0.0996
core factorial	Mamdani, 3 rules, 2 layer(s), window 20	2057	1.7282	0.0692
core factorial	FANTF+Mamdani, 5 rules, 2 layer(s), window 20	2123	1.7300	0.0312
core factorial	Mamdani, 3 rules, 2 layer(s), window 10	1977	1.7369	0.0580
core factorial	Mamdani, 3 rules, 1 layer(s), window 20	1129	1.7382	0.0550
core factorial	Mamdani, 7 rules, 2 layer(s), window 20	2185	1.7458	0.0904
core factorial	Mamdani, 5 rules, 2 layer(s), window 10	2041	1.7476	0.0448
core factorial	FANTF+Mamdani, 7 rules, 2 layer(s), window 20	2187	1.7562	0.0310
core factorial	Mamdani, 7 rules, 2 layer(s), window 10	2105	1.7688	0.0371
core factorial	FANTF+Mamdani, 5 rules, 2 layer(s), window 10	2043	1.7761	0.0667
core factorial	Mamdani, 5 rules, 2 layer(s), window 20	2121	1.8109	0.0334
curve pairs	parabolic -> gaussian	1009	1.6099	0.0470
curve pairs	trapezoidal -> triangular	1009	1.6120	0.0427
curve pairs	triangular -> gaussian	1009	1.6123	0.0456
curve pairs	trapezoidal -> parabolic	1009	1.6131	0.0449
curve pairs	gaussian -> parabolic	1009	1.6137	0.0451
curve pairs	gaussian -> triangular	1009	1.6146	0.0460
curve pairs	gaussian -> trapezoidal	1009	1.6160	0.0467
curve pairs	parabolic -> trapezoidal	1009	1.6165	0.0493
curve pairs	gaussian -> gaussian	1009	1.6166	0.0473
curve pairs	trapezoidal -> gaussian	1009	1.6175	0.0449
curve pairs	triangular -> trapezoidal	1009	1.6188	0.0530
curve pairs	trapezoidal -> trapezoidal	1009	1.6191	0.0489

Family	Configuration	Params	RMSE	SD
curve pairs	parabolic -> triangular	1009	1.6225	0.0542
curve pairs	parabolic -> parabolic	1009	1.6247	0.0608
curve pairs	triangular -> parabolic	1009	1.6262	0.0496
curve pairs	triangular -> triangular	1009	1.6268	0.0498
encoder depth	1 encoder layer(s)	1009	1.6225	0.0542
encoder depth	2 encoder layer(s)	1937	1.6471	0.0722
encoder depth	3 encoder layer(s)	2865	1.7656	0.0798
fantf delta	delta 0.1	1010	1.6141	0.0530
fantf delta	delta 0.25	1010	1.6234	0.0567
fantf delta	delta 0.5	1010	1.6365	0.0664
fantf delta	delta 1	1010	1.6386	0.0691
fuzzy placement	after_feedforward	1009	1.6225	0.0542
fuzzy placement	after_attention	1009	1.6296	0.0566
fuzzy placement	both	1065	1.6419	0.0583
sequence length	window 10	1049	1.5904	0.0476
sequence length	window 5	1009	1.6225	0.0542
sequence length	window 20	1129	1.7382	0.0550
sequence length	window 30	1209	1.7984	0.0722

Appendix B. Reproducibility and references

Primary commands

Purpose	Command
Full study	/Users/sam/anaconda3/bin/python3 run_comprehensive_study.py
Unit tests	/Users/sam/anaconda3/bin/python3 -m unittest discover -s tests -v
Build report	.venv/bin/python build_comprehensive_study_report.py

Evidence artifacts

Artifact	Path
Screening summary	outputs/comprehensive_study/screening_summary.csv
Raw screening folds	outputs/comprehensive_study/screening_runs.csv
Confirmation summary	outputs/comprehensive_study/confirmation_summary.csv
Raw confirmation folds	outputs/comprehensive_study/confirmation_runs.csv
Held-out seed metrics	outputs/comprehensive_study/heldout_summary.csv
Machine-readable synthesis	outputs/comprehensive_study/summary.json
Implementation	src/ierspop_transformer/model.py

References

#	Reference
1	Vaswani et al. Attention Is All You Need. arXiv:1706.03762.
2	Chakraborty and Heintz. Enhancing Time Series Forecasting with Fuzzy Attention-Integrated Transformers. arXiv:2504.00070; code: https://github.com/sanjaylopa22/FANTF
3	Nie et al. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. arXiv:2211.14730.
4	Liu et al. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. arXiv:2310.06625.
5	Mamdani and Assilian. An Experiment in Linguistic Synthesis with a Fuzzy Logic Controller. DOI: 10.1016/S0020-7373(75)80002-2.
6	Harvard NLP. The Annotated Transformer. https://github.com/harvardnlp/annotated-transformer

Reproducibility note: all generated summary tables are derived from committed CSV/JSON artifacts. The dataset itself is not committed to the repository and must be supplied separately.